Kernix benchmark report: 0.2.0

Measured 2026-10-19T14:33:41Z on 12th Gen Intel(R) Core(TM) i3-1215U.

Rust: rustc 1.99.0 (b940084d7 2026-09-28); NumPy: 2.5.4; one CPU thread; float64.

Source: `815518001653562c8881f986a439f65464eebeea`; working tree dirty: True.

Comparative CPU results

Workload	Kernix p50 / p95 (us)	ndarray p50 / p95 (us)	NumPy p50 / p95 (us)
add_65536_f64	26.131 / 38.754	24.535 / 32.052	22.794 / 24.893
matmul_128_f64	382.258 / 490.539	88.789 / 151.150	697.070 / 760.648
matmul_32_f64	5.921 / 7.797	1.551 / 2.836	10.537 / 13.784
matmul_reuse_128_f64	418.931 / 500.082	87.281 / 102.776	701.066 / 706.400
matmul_reuse_32_f64	5.529 / 5.798	1.500 / 1.671	11.144 / 11.545
softmax_256x64_f64	221.617 / 326.208	67.251 / 85.186	92.273 / 137.054
sum_65536_f64	33.711 / 35.444	8.789 / 8.895	10.351 / 10.768

Method and limits

Operations allocate outputs except scalar sums and explicitly named matmul_reuse cases. Reuse cases time preallocated output writes in all libraries. Inputs are preallocated and contiguous. ndarray uses its default matrixmultiply engine (no BLAS feature); NumPy uses the build recorded in raw.json. BLAS/OMP thread counts are fixed to one. These are warmed, sequential wall-clock samples on a shared laptop; CPU frequency and other system load are uncontrolled. p95 is the maximum of 15 batch means, not a per-request tail-latency guarantee. Different sum orders may round differently. The ndarray softmax is an explicitly coded stable rowwise comparison, not a native ndarray softmax API. NumPy timings include Python call overhead. No GPU comparison. No claim that a complete Full HD model reaches 120 FPS follows from these measurements. raw.json contains all measurements, dependencies and environment metadata.